PYTHON KURSU • HAFTA 10 • GÜN 1

NumPy 2: Matrisler ve Filtreleme

Ders 19 / 32 • 3 saat • VS Code + Python 3.12

Bu dersin sonunda…

- Matrislerle dört işlem yapıp yayma (broadcasting) kurallarını uygulayabileceksiniz.
- @ ile matris çarpımını gerçek bir hesapta (adet × fiyat) kullanabileceksiniz.
- Koşullarla filtreleyebilecek, & | ~ ile koşulları birleştirebileceksiniz.
- np.where, sort, argsort, cumsum ve diff ile veriyi dönüştürebileceksiniz.
- concatenate ve vstack ile dizileri birleştirip bir satış raporu hazırlayabileceksiniz.

AÇILIŞ • 10 DK

Bugünün zaman planı

Saat	Süre	Bölüm
0:00–0:10	10 dk	Açılış: hedefler, Ders 18'in tekrarı
0:10–1:25	75 dk	Blok 1: matrislerle dört işlem • yayma: satır ve sütun vektörü • transpoz • matris çarpımı @ • cumsum ve diff • eksik veri (NaN)
1:25–1:40	15 dk	Mola
1:40–2:55	75 dk	Blok 2: filtreleme • & | ~ • satır seçme • np.where • maske ile güncelleme • sıralama • birleştirme • kopya/görünüm • satış raporu • alıştırmalar
2:55–3:00	5 dk	Ödev ve gelecek ders

AÇILIŞ • DERS 18'IN TEKRARI • 6 DK

Dört soruda geçen ders

SORU 1 • Ne yazar?

```
np.array([1, 2, 3]) * 2
```

SORU 2 • Kaç sayı döner?

```
notlar.shape  # (4, 3)
notlar.mean(axis=1)
```

SORU 3 • Ne verir?

```
np.array([17, 22, 19]).argmax()
```

SORU 4 • Neden aynı?

```
rng = np.random.default_rng(42)
# her çalıştırmada aynı sayılar
```

AÇILIŞ • ÖDEV KONTROLÜ • 4 DK

Ödev 2: 6000 zar atışı

```python
import numpy as np

rng = np.random.default_rng(2026)
zarlar = rng.integers(1, 7, size=6000)
for yuz in range(1, 7):
    print(yuz, (zarlar == yuz).sum())
```

```
ÇIKTI
1 994
2 1020
3 1024
4 985
5 976
6 1001
```

BLOK 1 • 0:10–1:25 • 75 DK

Matrislerle hesap

20 dk
Dört işlem ve yayma

20 dk
Matris çarpımı

25 dk
cumsum, diff, NaN

10 dk
Hatalar

BLOK 1 • DÖRT IŞLEM VE YAYMA • 20 DK

Aynı şekilli iki matris: eleman eleman işlem

```
import numpy as np

bu_yil = np.array([[120, 135, 128, 150], [98, 110, 105, 120], [140, 138, 152, 160]])
gecen_yil = np.array([[110, 120, 125, 130], [100, 104, 98, 112], [130, 140, 135, 150]])
fark = bu_yil - gecen_yil
buyume = (fark / gecen_yil * 100).round(1)
print(fark)
print(buyume)
```

```
ÇIKTI
[[10 15  3 20]
 [-2  6  7  8]
 [10 -2 17 10]]
[[ 9.1 12.5  2.4 15.4]
 [-2.   5.8  7.1  7.1]
 [ 7.7 -1.4 12.6  6.7]]
```

Satırlar mağazalar, sütunlar aylar. **Büyüme %** = (bu yıl – geçen yıl) / geçen yıl × 100.

BLOK 1 • DÖRT IŞLEM VE YAYMA • 20 DK

Yayma (1): her sütuna bir değer

```
hedef_ay = np.array([125, 125, 130, 140])      # her ay için hedef
print(satis - hedef_ay)
```

```
ÇIKTI
[[ -5  10  -2  10]
 [-27 -15 -25 -20]
 [ 15  13  22  20]]
```

Benzetme: aynı cetveli her satıra koymak

(3, 4) matris ile (4,) dizi: NumPy hedef dizisini her mağaza satırının altına kopyalanmış gibi kullanır. Satır uzunlukları (4) eşleştiği için işlem yapılabilir.

Yayma (2): her satıra bir değer

```
hedef_magaza = np.array([130, 105, 145]).reshape(-1, 1)   # (3, 1)
print(hedef_magaza.shape)
print(satis - hedef_magaza)
```

```
ÇIKTI
(3, 1)
[[-10   5  -2  20]
 [ -7   5   0  15]
 [ -5  -7   7  15]]
```

Şekiller

`(3, 4) ile (4,)`
`(3, 4) ile (3, 1)`
`(3, 4) ile (3,)`

Sonuç

dizi her SATIRA uygulanır (ay bazında)
dizi her SÜTUNA uygulanır (mağaza bazında)
hata: son boyutlar 4 ≠ 3

BLOK 1 • DÖRT IŞLEM VE YAYMA • 20 DK

.T: satırları sütun yapmak

```
print(satis.T)
print(satis.T.shape)
```

```
ÇIKTI
[[120  98 140]
 [135 110 138]
 [128 105 152]
 [150 120 160]]
(4, 3)
```

Satırlar artık aylar, sütunlar mağazalar. Tabloyu “çevirmek” bazı hesapları ve grafikleri kolaylaştırır.

BLOK 1 • MATRIS ÇARPIMI • 20 DK

Matris çarpımı @: adet × fiyat = ciro

```
import numpy as np

urunler = np.array(["Defter", "Kalem", "Çanta"])
fiyat = np.array([45.0, 12.5, 650.0])          # ürün başına TL
# satırlar: mağazalar · sütunlar: ürünlerden satılan adet
adet = np.array([[30, 120, 4],
                 [25,  80, 6],
                 [40, 150, 2]])
ciro = adet @ fiyat
print(ciro)
print("Elle (Kadıköy):", 30 * 45.0 + 120 * 12.5 + 4 * 650.0)
```

```
ÇIKTI
[5450. 6025. 4975.]
Elle (Kadıköy): 5450.0
```

Kural

(3, 3) @ (3,) → her satır, fiyat dizisiyle çarpılıp toplanır. İç boyutlar eşit olmalı.

BLOK 1 • MATRIS ÇARPIMI • 20 DK

Ders 18’deki dönem notu: @ ile tek işlem

```python
import numpy as np

notlar = np.array([[60, 80, 70], [75, 90, 85], [40, 65, 55]])
agirlik = np.array([0.3, 0.2, 0.5])
print((notlar * agirlik).sum(axis=1))
print(notlar @ agirlik)
```

```
ÇIKTI
[69.  83.  52.5]
[69.  83.  52.5]
```

İşlem	Yaptığı	Şekil örneği
`a * b`	eleman eleman (yayma ile)	`(3, 3) * (3,) → (3, 3)`
`a @ b`	satır × sütun, toplanır	`(3, 3) @ (3,) → (3,)`
`A @ B (iki matris)`	her satır × her sütun	`(3, 2) @ (2, 4) → (3, 4)`

BLOK 1 • CUMSUM, DIFF, NAN • 25 DK

cumsum ve diff: birikim ve değişim

```
kadikoy = satis[0]
print("Aylık:", kadikoy)
print("Kümülatif:", kadikoy.cumsum())
print("Aydan aya fark:", np.diff(kadikoy))
print("Aylık büyüme %:", (np.diff(kadikoy) / kadikoy[:-1] * 100).round(1))
print("Mağaza kümülatifleri:")
print(satis.cumsum(axis=1))
```

```
ÇIKTI
Aylık: [120 135 128 150]
Kümülatif: [120 255 383 533]
Aydan aya fark: [15 -7 22]
Aylık büyüme %: [12.5 -5.2 17.2]
Mağaza kümülatifleri:
[[120 255 383 533]
 [ 98 208 313 433]
 [140 278 430 590]]
```

BLOK 1 • CUMSUM, DIFF, NAN • 25 DK

Eksik veri: NaN (Not a Number)

```
import numpy as np

olcum = np.array([21.5, np.nan, 23.0, 22.4, np.nan])
print(olcum.mean())                 # nan "bulaşıcıdır"
print(np.nanmean(olcum).round(2))   # nan'ları yok say
print(np.isnan(olcum), np.isnan(olcum).sum(), "eksik")
```

```
ÇIKTI
nan
22.3
[False  True False False  True] 2 eksik
```

Benzetme

Cihaz bir gün bozulmuş, defterde o gün boş. O günü 0 yazmak ortalamayı düşürür; “bilinmiyor” (NaN) yazmak dürüsttür.

BLOK 1 • SIK YAPILAN HATALAR

Matris hesaplarında sık hatalar

Hatalı kod	Hata mesajı	Çözüm
`satis (3,4) - np.array([…3 eleman])`	`ValueError: operands could not be broadcast together with shapes (3,4) (3,)`	reshape(-1, 1)
`adet (3,3) @ fiyat (4,)`	`ValueError: matmul: Input operand 1 has a mismatch in its core dimension 0, with gufunc signature (n?,k),(k,m?)→(n?,m?) (size 4 is different from 3)`	iç boyutlar eşit
`dizi @ 3`	`ValueError: matmul: Input operand 1 does not have enough dimensions (has 0, gufunc core with signature (n?,k),(k,m?)→(n?,m?) requires 1)`	sayıyla * kullanın

Hata vermeyen tuzak	Doğrusu
`a * b` yazıp matris çarpımı sanmak	`a @ b`
`olcum == np.nan` → hep False	`np.isnan(olcum)`
NaN içeren dizide `mean()` → nan	`np.nanmean(…)`

1:25–1:40 • 15 DK

Mola

Moladan sonra: hangi mağaza hangi ayda hedefi tutturdu? Tek satırda.

BLOK 2 • 1:40–2:55 • 75 DK

Seçmek, sıralamak, birleştirmek

25 dk
Filtreleme

15 dk
Sıralama ve birleştirme

17 dk
Satış raporu

18 dk
Alıştırmalar

BLOK 2 • FILTRELEME • 25 DK

Filtreleme: dizi[koşul]

```
import numpy as np

kadikoy = np.array([120, 135, 128, 150])
aylar = np.array(["Oca", "Şub", "Mar", "Nis"])
maske = kadikoy > 130
print(maske)
print(kadikoy[maske])
print(aylar[maske])
print(kadikoy[kadikoy < 125])
```

```
ÇIKTI
[False  True False  True]
[135 150]
['Şub' 'Nis']
[120]
```

Benzetme: kevgir

Maske bir kevgirin delikleri gibidir: True olan konumlardaki değerler geçer, False olanlar takılır. Aynı maske başka bir diziye (ay adları) de uygulanabilir.

BLOK 2 • FILTRELEME • 25 DK

Koşulları birleştirmek: & (ve), | (veya), ~ (değil)

```python
import numpy as np

kadikoy = np.array([120, 135, 128, 150])
print(kadikoy[(kadikoy > 125) & (kadikoy < 140)])   # ve
print(kadikoy[(kadikoy < 125) | (kadikoy > 140)])   # veya
print(kadikoy[~(kadikoy > 130)])                    # değil
```

```
ÇIKTI
[135 128]
[120 150]
[120 128]
```

Python (tek değer)	NumPy (dizi)	Not
`and`	`&`	her koşul parantez içinde
`or`	`|`	her koşul parantez içinde
`not`	`~`	~(koşul)

BLOK 2 • FILTRELEME • 25 DK

Matriste filtreleme: değer seçmek ve satır seçmek

```
print(satis[satis > 130])                    # 2B maske → 1B sonuç
print((satis > 130).sum(axis=1))             # mağaza başına kaç ay
print(magazalar[satis[:, 3] > 130])          # Nisan'da 130'u geçenler
print(satis[satis.sum(axis=1) > 480])        # toplamı 480'i geçen mağazalar
```

```
ÇIKTI
[135 150 140 138 152 160]
[2 0 4]
['Kadıköy' 'Çankaya']
[[120 135 128 150]
 [140 138 152 160]]
```

BLOK 2 • FILTRELEME • 25 DK

np.where: koşula göre değer seçmek

```
hedef = 125
durum = np.where(satis ⩾ hedef, "Tuttu", "Kaldı")
print(durum)
duzeltilmis = np.where(satis > 150, 150, satis)      # üst sınır
print(duzeltilmis)
satirlar, sutunlar = np.where(satis < 105)
print("Hedefin çok altı:", magazalar[satirlar], aylar[sutunlar])
```

```
ÇIKTI
[['Kaldı' 'Tuttu' 'Tuttu' 'Tuttu']
 ['Kaldı' 'Kaldı' 'Kaldı' 'Kaldı']
['Tuttu' 'Tuttu' 'Tuttu' 'Tuttu']]
[[120 135 128 150]
 [ 98 110 105 120]
 [140 138 150 150]]
Hedefin çok altı: ['Bornova']
  ['Oca']
```

BLOK 2 • FILTRELEME • 25 DK

Maske ile atama: hatalı değerleri temizlemek

```
import numpy as np

olcum = np.array([21.5, -999.0, 23.0, 22.4, 85.0])   # -999: sensör hatası kodu
olcum[olcum == -999] = np.nan
olcum[olcum > 50] = np.nan                            # imkânsız değer
print(olcum)
print("Ortalama:", np.nanmean(olcum).round(2))
```

```
ÇIKTI
[21.5  nan 23.  22.4  nan]
Ortalama: 22.3
```

BLOK 2 • SIRALAMA VE BIRLEŞTIRME • 15 DK

sort ve argsort: sıralamak ve sıraya göre dizmek

```
toplam = satis.sum(axis=1)
print(np.sort(toplam))                  # küçükten büyüğe
print(np.sort(toplam)[::-1])            # büyükten küçüğe
sira = np.argsort(toplam)[::-1]         # büyükten küçüğe KONUMLAR
print(sira, magazalar[sira])
print(satis[sira])                      # satırları toplama göre sırala
```

```
ÇIKTI
[433 533 590]
[590 533 433]
[2 0 1] ['Çankaya' 'Kadıköy'
  'Bornova']
[[140 138 152 160]
 [120 135 128 150]
 [ 98 110 105 120]]
```

BLOK 2 • SIRALAMA VE BIRLEŞTIRME • 15 DK

Birleştirmek: yeni mağaza (satır), yeni ay (sütun)

```
izmit = np.array([[105, 112, 118, 125]])          # yeni mağaza (1, 4)
satis2 = np.vstack([satis, izmit])
magazalar2 = np.append(magazalar, "İzmit")
print(satis2.shape, magazalar2)
mayis = np.array([155, 118, 165, 130]).reshape(-1, 1)   # yeni ay (4, 1)
satis3 = np.hstack([satis2, mayis])
print(satis3.shape)
print(satis3)
```

```
ÇIKTI
(4, 4) ['Kadıköy' 'Bornova'
  'Çankaya' 'İzmit']
(4, 5)
[[120 135 128 150 155]
 [ 98 110 105 120 118]
 [140 138 152 160 165]
 [105 112 118 125 130]]
```

Fonksiyon	Ekler
`np.vstack`	alta satır
`np.hstack`	yana sütun
`np.concatenate(axis=…)`	genel

BLOK 2 • SIRALAMA VE BIRLEŞTIRME • 15 DK

Dilim bir kopya değil, görünümdür

```
import numpy as np

satis = np.array([120, 135, 128, 150])
ilk_iki = satis[:2]          # görünüm (view)
ilk_iki[0] = 0
print(satis)                 # orijinal de değişti!
yedek = satis.copy()         # bağımsız kopya
yedek[1] = 999
print(satis)
print(yedek)
```

```
ÇIKTI
[  0 135 128 150]
[  0 135 128 150]
[  0 999 128 150]
```

Kural

Dilimi değiştirecekseniz ve orijinalin bozulmasını istemiyorsanız .copy() kullanın. (Maske ile seçim zaten kopya üretir.)

BLOK 2 • SATIŞ RAPORU • 17 DK

Uygulama: dört aylık satış raporu

```
Rapor (veri tanımları .py dosyasında)
toplam = satis.sum(axis=1)
buyume = ((toplam - gecen_yil.sum(axis=1)) / gecen_yil.sum(axis=1) * 100).round(1)
hedef_ay_sayisi = (satis ⩾ hedef).sum(axis=1)       # hedef: (3, 1)
en_iyi_ay = aylar[satis.argmax(axis=1)]

for i in np.argsort(toplam)[::-1]:                  # büyükten küçüğe
    print(f"{magazalar[i]:<9}{toplam[i]:>7}{buyume[i]:>9}"
          f"{hedef_ay_sayisi[i]:>5}/4  {en_iyi_ay[i]}")
```

YAYMA

satis >= hedef

(3, 4) ile (3, 1): her mağaza kendi hedefiyle.

SATIR BAŞINA

argmax(axis=1)

Her mağazanın en iyi ayının konumu → ay adı.

SIRALAMA

argsort()[::-1]

Rapor, toplam satışa göre büyükten küçüğe.

BLOK 2 • SATIŞ RAPORU • 17 DK

Raporun çıktısı

```
ÇIKTI
Mağaza    Toplam  Büyüme%  Hedef  En iyi ay
Çankaya      590      6.3    2/4  Nis
Kadıköy      533      9.9    2/4  Nis
Bornova      433      4.6    3/4  Nis
Zincir toplamı: 1556 · Düşüş görülen hücre sayısı: 2
```

Aynı raporu gelecek ders Pandas ile, mağaza ve ay adları tablonun içinde olacak şekilde yeniden yazacağız.

BLOK 2 • SIK YAPILAN HATALAR

Filtreleme ve birleştirmede sık hatalar

Hatalı kod	Hata mesajı	Çözüm
`a[(a > 125) and (a < 140)]`	`ValueError: The truth value of an array with more than one element is ambiguous. Use a.any() or a.all()`	& kullanın
`a[a > 125 & a < 140]`	`ValueError: The truth value of an array with more than one element is ambiguous. Use a.any() or a.all()`	parantez: (a > 125) & (a < 140)
`np.vstack([satis, [1, 2, 3]])`	`ValueError: all the input array dimensions except for the concatenation axis must match exactly, but along dimension 1, the array at index 0 has size 4 and the array at index 1 has size 3`	sütun sayısı 4 olmalı
`int_dizi[0] = np.nan`	`ValueError: cannot convert float NaN to integer`	float dizi: astype(float)

BLOK 2 • ÖZET

NumPy’de iki derste öğrendiklerimiz

İş	Araç
Oluşturmak	`np.array · arange · linspace · zeros · ones · full · default_rng`
Bilgi	`.shape · .dtype · .ndim · .size`
Hesap	`+ - * / ** (yayma ile) · @ · np.sqrt · np.round`
Özet	`sum · mean · min · max · std · argmax · argmin · cumsum · np.diff (axis=…)`
Seçmek	`d[i] · d[a:b] · m[s, k] · d[maske] · & | ~ · np.where`
Düzenlemek	`reshape · .T · np.sort · argsort · vstack · hstack · .copy()`
Eksik veri	`np.nan · np.isnan · np.nanmean`

BLOK 2 • SINIF IÇI ALIŞTIRMALAR • 18 DK

Alıştırma 1, 2 ve 3

KOLAY • 4 dk

1 • Sıcak günler

Haftalık sıcaklık dizisinde 20 °C üstü günlerin adlarını ve sıcaklıklarını; 18–21 arası gün sayısını bulun.

ORTA • 6 dk

2 • Sınav temizliği

Not dizisinde -1 “sınava girmedi” demek. Bunları NaN yapıp ortalamayı bulun; girmeyenlerin sayısı ve np.where ile “Geçti/Kaldı/Girmedi” etiketleri.

ZOR • 8 dk

3 • Ürün cirosu

Mağaza × ürün adet matrisi ve fiyat dizisiyle: mağaza cirosu (@), ürün başına toplam adet, en çok ciro yapan mağaza, cirosu ortalamanın üstündeki mağazalar.

BLOK 2 • ALIŞTIRMA ÇÖZÜMLERI

Çözüm 1: sıcak günler

```
import numpy as np

gunler = np.array(["Pzt", "Sal", "Çar", "Per", "Cum", "Cmt", "Paz"])
sicaklik = np.array([19.8, 18.1, 16.5, 21.2, 22.0, 20.3, 17.9])
sicak = sicaklik > 20
print(gunler[sicak], sicaklik[sicak])
print(((sicaklik ⩾ 18) & (sicaklik ⩽ 21)).sum(), "gün 18-21 °C")
```

```
ÇIKTI
['Per' 'Cum' 'Cmt'] [21.2 22.  20.3]
3 gün 18-21 °C
```

BLOK 2 • ALIŞTIRMA ÇÖZÜMLERI

Çözüm 2: sınav temizliği

```
import numpy as np

notlar = np.array([72, -1, 45, 88, -1, 60, 38]).astype(float)
girmedi = notlar == -1
notlar[girmedi] = np.nan
print("Ortalama:", np.nanmean(notlar).round(1), "· Girmeyen:", girmedi.sum())
etiket = np.where(girmedi, "Girmedi", np.where(notlar ≥ 50, "Geçti", "Kaldı"))
print(etiket)
```

```
ÇIKTI
Ortalama: 60.6 · Girmeyen: 2
['Geçti' 'Girmedi' 'Kaldı' 'Geçti' 'Girmedi' 'Geçti' 'Kaldı']
```

BLOK 2 • ALIŞTIRMA ÇÖZÜMLERI

Çözüm 3: ürün cirosu

```python
import numpy as np

magazalar = np.array(["Kadıköy", "Bornova", "Çankaya"])
fiyat = np.array([45.0, 12.5, 650.0])
adet = np.array([[30, 120, 4], [25, 80, 6], [40, 150, 2]])
ciro = adet @ fiyat
print("Ciro:", ciro)
print("Ürün başına adet:", adet.sum(axis=0))
print("En çok ciro:", magazalar[ciro.argmax()])
print("Ortalamanın üstü:", magazalar[ciro > ciro.mean()])
```

```
ÇIKTI
Ciro: [5450. 6025. 4975.]
Ürün başına adet: [ 95 350  12]
En çok ciro: Bornova
Ortalamanın üstü: ['Bornova']
```

KAPANIŞ • 5 DK

Bugün ne öğrendik?

- Aynı şekilli matrisler eleman eleman; farklı şekillerde **yayma**: (3, 4) ile (4,) satıra, (3, 1) sütuna.
- **@** matris çarpımı: adet @ fiyat = ciro; ***** ile karıştırmayın.
- **cumsum** birikim, **np.diff** değişim; eksik veri **np.nan** + **nanmean**.
- Filtre: **d[koşul]**; birleştirme **(…) & (…)**, **|**, **~**; **np.where(k, a, b)**.
- **sort/argsort** sıralar; **vstack/hstack** birleştirir; dilim görünümdür, **.copy()** kopyadır.

KAPANIŞ • ÖDEV • TESLIM: BIR SONRAKI DERS

Ödev

1 • ZORUNLU

Kantin satışları

5 ürün × 5 gün adet matrisi ve fiyat dizisi: günlük ciro (@), en çok satan ürün, hiç satılmayan gün/ürün sayısı, ciroya göre sıralı gün listesi.

2 • ZORUNLU

Sensör temizliği

default_rng(5) ile 30 günlük sıcaklık üretin (normal(20, 4)), 3 rastgele güne -999 yazın. Temizleyip nanmean, en sıcak gün ve 3 günlük farkları (diff) bulun.

3 • BONUS

Pandas'a hazırlık

python -m pip install pandas openpyxl ile kurun; bugünkü satis matrisini pd.DataFrame(satis, index=magazalar, columns=aylar) ile tabloya çevirip yazdırın.

Teslim: ders19_odev.py ya da .ipynb. Teslim yöntemi: [teslim yöntemi]

KAPANIŞ • GELECEK DERS: HAFTA 10 • GÜN 2

Pandas 1: tablolarla tanışma

NumPy’de veri bir yerde, mağaza ve ay adları başka yerdeydi. Pandas ikisini tek tabloda birleştiriyor:

- Pandas nedir, NumPy’den farkı
- Series ve DataFrame
- index ve columns
- head, info, describe gibi temel metotlar

Önizleme

```
df = pd.DataFrame(satis,
    index=["Kadıköy", "Bornova", "Çankaya"],
    columns=["Oca", "Şub", "Mar", "Nis"])
print(df)
```

```
          Oca  Şub  Mar  Nis
Kadıköy   120  135  128  150
Bornova    98  110  105  120
Çankaya   140  138  152  160
```

KAYNAKLAR

Bu dersin konuları resmi belgelerde

Ne için?	Adres
Boolean indexing	numpy.org/doc/stable/user/basics.indexing.html
Broadcasting	numpy.org/doc/stable/user/basics.broadcasting.html
Birleştirme fonksiyonları	numpy.org/…/routines.array-manipulation.html
Kopya ve görünüm	numpy.org/doc/stable/user/basics.copies.html